

Sri Lanka Institute of Information Technology



IT2130 – Operating Systems and System Administration
Year 2, Semester 2- 2026

Practical Answer Submission

Practical Sheet No: 06

Student ID	IT24104150
Student Name	Sathsaranee K.L.U
Campus/ Center name	Malabe
Specialization	CSNE
Batch No	Y2S2.WD.CSNE.01

Activity 01

a)

```
GNU nano 7.2 lab6.c
#include <stdio.h>
#include <unistd.h>
#include <string.h>
#include <sys/wait.h>

int main() {
    int fd[2];
    pid_t pid;
    char write_msg[] = "Hello!";
    char read_msg[100];

    pipe(fd);
    pid = fork();

    if (pid > 0) {
        close(fd[0]);
        write(fd[1], write_msg, strlen(write_msg) + 1);
        close(fd[1]);
        wait(NULL);
    } else {
        close(fd[1]);
        read(fd[0], read_msg, sizeof(read_msg));
        printf("Child received: %s\n", read_msg);
        close(fd[0]);
    }

    return 0;
}
```

```
vboxuser@ubuntu:~$ nano lab6.c
vboxuser@ubuntu:~$ gcc -o lab6 lab6.c
vboxuser@ubuntu:~$ ./lab6
Child received: Hello!
```

b) Line A – pipe(fd);

- Creates a pipe.
- fd[0] → Read end
- fd[1] → Write end

Line B – fork();

- Creates a child process.
- After this line:
- Parent and child both continue execution.
- Return value:

- 0 → Child
- 0 → Parent

Line C – `write(fd[1], message, sizeof(message));`

- Writes data into the pipe.
- Usually done by the parent.
- Sends message through `fd[1]` (write end).

Line D – `read(fd[0], buffer, sizeof(buffer));`

- Reads data from the pipe.
- Usually done by the child.
- Reads message from `fd[0]` (read end).

Activity 02

```
GNU nano 7.2 lab6.c
#include <stdio.h>
#include <unistd.h>
#include <string.h>

int main() {
    int fd[2];
    char message[] = "Hello Parent";
    char buffer[100];

    pipe(fd);
    int pid = fork();

    if (pid == 0) {
        close(fd[0]);
        write(fd[1], message, strlen(message) + 1);
        close(fd[1]);
    } else {
        close(fd[1]);
        read(fd[0], buffer, sizeof(buffer));
        printf("Parent received: %s\n", buffer);
        close(fd[0]);
    }

    return 0;
}
```

```
vboxuser@ubuntu:~$ nano lab6.c
vboxuser@ubuntu:~$ gcc -o lab6 lab6.c
vboxuser@ubuntu:~$ ./lab6
Parent received: Hello Parent
```

Activity 03

```
GNU nano 7.2 lab6.c
#include <stdio.h>
#include <unistd.h>

int main() {
    int pipe1[2], pipe2[2];
    int number, result;

    pipe(pipe1);
    pipe(pipe2);

    int pid = fork();

    if (pid == 0) {
        close(pipe1[1]);
        close(pipe2[0]);

        read(pipe1[0], &number, sizeof(number));
        result = number * 2;
        write(pipe2[1], &result, sizeof(result));

        close(pipe1[0]);
        close(pipe2[1]);
    } else {
        close(pipe1[0]);
        close(pipe2[1]);

        number = 5;
        write(pipe1[1], &number, sizeof(number));
```

```
        number = 5;
        write(pipe1[1], &number, sizeof(number));

        read(pipe2[0], &result, sizeof(result));
        printf("Result received from child: %d\n", result);

        close(pipe1[1]);
        close(pipe2[0]);
    }

    return 0;
}
```

```
vboxuser@ubuntu:~$ nano lab6.c
vboxuser@ubuntu:~$ gcc -o lab6 lab6.c
vboxuser@ubuntu:~$ ./lab6
Result received from child: 10
vboxuser@ubuntu:~$
```

